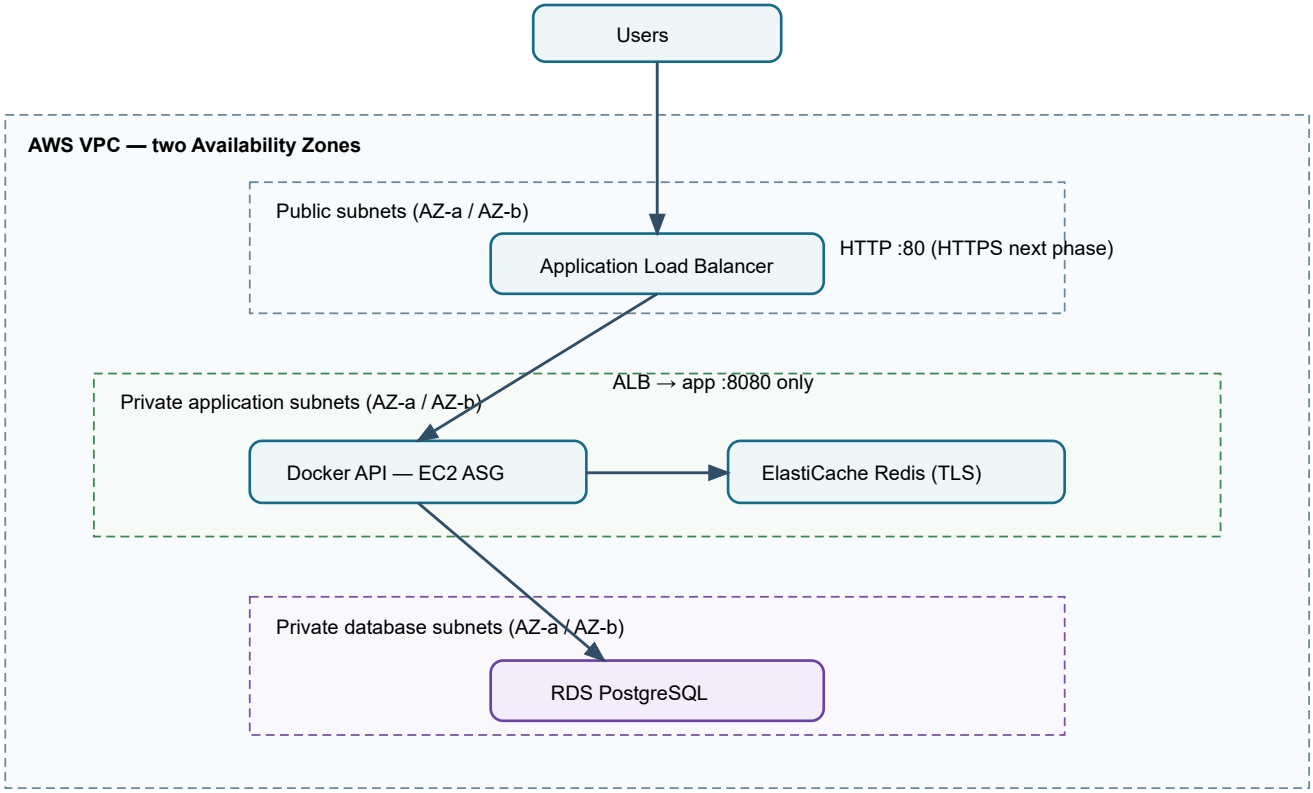
AWS Docker Deployment Platform

Architecture document · Senior DevOps Assignment · AWS-focused implementation

> **Scope statement:** This is a defendable AWS implementation based on VPC networking, EC2 Auto Scaling, Docker, RDS, Redis and an Application Load Balancer. Kubernetes and multi-cloud are proposed roadmap items and are not claimed as implemented.

1. Architecture overview

Terraform creates one VPC in two Availability Zones. The ALB is the only internet-facing workload. Docker application instances run in private application subnets and accept port 8080 only from the ALB security group. PostgreSQL and Redis are private and accept traffic only from the application security group.



Implemented: VPC, six subnets, IGW, NAT Gateway, route tables, security groups, ALB, ASG, RDS, Redis, Secrets Manager and immutable container image reference. **Proposed:** EKS/kind, GitOps, metric-gated canary, tenant fleet controller and second cloud.

2. Architecture decisions and rejected alternatives

#	Decision	Reason	Rejected alternative
1	**AWS-first implementation**	Matches skills that can be demonstrated and operated within the assignment timebox.	Provider-neutral Kubernetes implementation was rejected for this phase because it could not be honestly tested and defended. EKS plus local kind is the next step.
2	**EC2 Auto Scaling with Docker**	Provides horizontal scaling and replacement while keeping the runtime understandable.	EKS was rejected for the implemented phase because it adds cluster lifecycle, CNI, ingress, workload identity, HPA/KEDA and PDB operations.
3	**Rolling instance refresh**	ASG maintains 100% minimum healthy capacity; the ALB sends traffic only to healthy new instances.	Blue/green was rejected because it duplicates the compute fleet and requires safe listener switching. Canary is proposed after metric gates exist.
4	**Managed RDS and ElastiCache**	Managed patching, backup, encryption and failover reduce operational burden.	Self-managed PostgreSQL and Redis on EC2 were rejected because the team would own replication, recovery, patching and failover.
5	**Private application and database tiers**	Only the ALB is public. Instances use Systems Manager; PostgreSQL and Redis allow only the app security group.	Public instances, direct SSH and a public database were rejected because they increase attack surface.
6	**One NAT Gateway for the demo**	Controls cost for the initial implementation.	One NAT per AZ is safer but costs more. It is the production recommendation because the current NAT creates an AZ dependency.
7	**Build once, deploy by digest**	The exact scanned bytes move between environments; production rejects tag-only image references.	Rebuilding per environment was rejected because different artifacts could reach staging and production.

3. Security and data boundaries

- ALB security group permits public HTTP; application instances permit port 8080 only from the ALB.
- RDS permits 5432 and Redis permits 6379 only from the application security group.
- Database credentials are generated and stored in Secrets Manager. EC2 receives least-privilege read access.
- IMDSv2 is mandatory, storage is encrypted, Redis uses in-transit encryption, and RDS is not public.
- EU tenant compute, data, backups, logs and metrics must remain in approved EU regions. Organization policies are proposed to deny non-EU placement.

4. Delivery and zero-downtime approach

Implemented release path

GitHub Actions builds the application image once, scans it with Trivy, pushes it to ECR and resolves the immutable SHA-256 digest. Terraform creates a new launch-template version and starts an ASG rolling refresh. New instances receive traffic only after `/healthz` passes. ALB deregistration is 60 seconds and Docker stop timeout is 75 seconds.

Proposed metric-gated canary

Traffic	Observation	Promotion/rollback gates
5%	5 minutes	Promote only when p95 latency is below 400 ms, HTTP 5xx is below 1%, readiness is at least 99%, and error-budget burn is acceptable. Automatically return traffic to the last known-good digest when a gate fails.
25%	10 minutes	
50%	10 minutes	
100%	Continuous	

Proposed rollout order: shared development → shared staging → shared production → internal dedicated tenants → low-risk dedicated tenants → high-value tenants → additional regions. A failed ring stops later rings.

Workers and database migrations

Workers should stop polling on termination, finish the active job for up to 60 seconds, acknowledge only after success, then exit. Scale workers from Redis queue depth and oldest-job age, not CPU alone.

Non-backward-compatible startup migrations prevent guaranteed zero downtime. Use expand/contract:

1. Add a schema change that is compatible with both application versions.
2. Deploy code capable of reading/writing both old and new forms.
3. Backfill and validate, then switch reads to the new form.
4. Remove the old schema only in a later release after rollback is no longer needed.

A binary rollback is safe only before the contract step. After a destructive migration, restore or forward-fix is required.

5. Multi-deployment model

One Terraform template is parameterized for shared or dedicated environments. The complete design uses one isolated remote state and secrets boundary per tenant. A reviewed tenant manifest selects tenant ID, region, size and shared/dedicated mode; automation plans, applies and validates it in under 30 minutes. Fleet rollout uses rings and halts on the first failed health/SLO gate.

6. Scaling, reliability and operations

API capacity

Required instances = `ceil(peak RPS ÷ tested safe RPS per instance × 1.30)`. If testing proves 150 RPS per instance below 400 ms p95, peak capacity is `ceil(2250 ÷ 150 × 1.30) = 20` instances. The ASG maximum is 30 for headroom. This assumption must be replaced with test evidence. Predictive scaling, a warm pool or baked AMI is needed to reliably reach 15× within five minutes.

Worker and database capacity

20,000 jobs/hour is 5.56 jobs/second. At the worst-case 60 seconds per job, about 334 concurrent worker slots are required before headroom. Real scaling uses observed duration, queue depth and oldest-job age. Scale-in must never terminate a consumer with unacknowledged work.

RDS is the likely stateful bottleneck. Monitor CPU, free memory, I/O latency, queue depth, locks and connections. Bound connection pools so `instances × pool size` remains under the database connection budget. Optimize queries/indexes before vertical scaling or read replicas; writes and migrations still use the primary.

SLOs, alerts and recovery

Area	Design
SLO	99.9% shared (≈43.8 min monthly error budget); 99.95% dedicated (≈21.9 min).
Alerts	Latency, 5xx, unhealthy targets, ASG capacity, queue age/depth, failed jobs, RDS CPU/connections/storage, backup failure and certificate expiry.
Visibility	Metrics and structured logs include environment and tenant ID, but exclude credentials and personal data.
Backup	RDS automated backup for 14 days; Redis snapshots for 7 days; copies remain inside the tenant's legal region.
Restore test	Quarterly isolated restore, integrity/schema checks and measured RPO/RTO. Proposed targets: RPO ≤5 minutes, RTO ≤60 minutes.

Cost controls

Steady state includes two EC2 instances, ALB, NAT/data processing, Multi-AZ RDS, two Redis nodes, storage, backups, secrets and monitoring. Peak adds about 18 API instances in the worked example plus stateful scaling. Validate with the AWS Pricing Calculator before review. Use budget alerts at 50/80/100%, mandatory cost tags, log retention and removal of idle non-production stacks.

7. Top five risks

#	Risk	Impact	Mitigation
1	**Unsafe startup migration**	Old instances can fail against a changed schema, causing downtime or making rollback impossible.	Enforce expand/contract, separate migration approval and compatibility testing.
2	**Single NAT Gateway**	An AZ failure can block image pulls and outbound AWS API access from private instances.	Use one NAT per AZ in production and add VPC endpoints for ECR, S3, SSM, logs and Secrets Manager.
3	**Slow EC2 cold start**	Boot, package installation and image pull may miss the 15×-in-five-minutes target.	Use predictive scaling, warm pools or a baked AMI; test and record scale-up time.
4	**PostgreSQL saturation**	Connections or I/O can fail before application compute reaches its ASG limit.	Bound pools, load-test database limits, tune queries/indexes and alert before saturation.
5	**AWS-only implementation gap**	The current implementation does not satisfy the complete cloud-agnostic Kubernetes requirement.	Label the limitation honestly; implement Helm on EKS and kind, then prove a second provider before claiming portability.

8. 90-day roadmap

- **Days 1–30:** deploy the AWS slice in a sandbox; add HTTPS/ACM, WAF, VPC endpoints, CloudWatch dashboards/alarms, queue metrics, automated restore tests and real load/rollback evidence.
- **Days 31–60:** implement Helm on local kind and AWS EKS with probes, PDBs, HPA/KEDA, External Secrets, Prometheus, Argo CD, signed images and metric-gated canaries.
- **Days 61–90:** add tenant manifests, rollout rings, EU organization policies, per-tenant observability, disaster-recovery exercises, verified pricing and a second-cloud proof of concept.

9. Portability statement

Portable concepts are the container image, health endpoints, immutable promotion, deployment gates, scaling signals, SLOs and expand/contract migration process. AWS-specific elements are VPC/subnets, ALB, ASG, IAM, Systems Manager, Secrets Manager, RDS, ElastiCache and CloudWatch. The future provider layer must map these capabilities rather than claiming identical resources across clouds.

> **Evidence status:** Terraform configuration was formatted and validated locally. AWS deployment, zero-error load test, automatic rollback, 15× scaling and restore test have not been run and must not be claimed until evidence is produced.